

Java OOP Interview Master Guide

20 ESSENTIAL QUESTIONS & ANSWERS FOR TECHNICAL SCREENINGS

| 1. Core OOP Concepts (The Big Four)

Q1. What are the four main pillars of OOP? How are they implemented in Java?

The four pillars form the foundation of Object-Oriented Programming:

- **Abstraction:** Hiding complex implementation details and showing only the essential features. Implemented using `abstract classes` and `interfaces`.
- **Encapsulation:** Binding data (variables) and code (methods) together into a single unit and restricting direct access. Implemented using `private` modifiers with public getters/setters.
- **Inheritance:** The mechanism by which one class acquires the properties and behaviors of another parent class. Implemented using the `extends` keyword.
- **Polymorphism:** The ability of an object to take on many forms. Implemented via Method Overloading (compile-time) and Method Overriding (runtime).

Q2. What is the difference between Abstraction and Encapsulation?

Abstraction focuses on *what* the object does rather than *how* it does it (design level). It solves problems at the interface level. **Encapsulation** focuses on *how* to hide the internal state of the object to secure it (implementation level). It isolates data logic strictly inside the class block using private modifiers.

Q3. What is Polymorphism? Can you explain its types?

Polymorphism translates to "many forms." In Java, it manifests in two distinct variants:

- **Compile-time (Static) Polymorphism:** Resolved during compilation. Achieved through **Method Overloading** (same method name, different argument structures within the same class context).
- **Runtime (Dynamic) Polymorphism:** Resolved dynamically at runtime. Achieved through **Method Overriding** (a subclass provides a specific implementation of a method already declared in its parent class hierarchy).

Q4. What is the difference between Method Overloading and Method Overriding?

Feature	Method Overloading	Method Overriding
Binding Time	Compile-time (Static binding)	Runtime (Dynamic binding)
--Location--	Happens within the same class definition.	Happens across inheritance structures (Parent/Child).
Signature	Must possess distinct argument profiles.	Must feature exact matches for arguments and types.
Private/Static	Perfectly valid to overload private/static items.	Cannot override methods designated private or static.

2. Classes, Interfaces, & Inheritance

Q5. Why doesn't Java support multiple inheritance with classes?

To prevent structural ambiguity known as the **Diamond Problem**. If Class A defines a method, and Classes B and C both inherit from A while overriding that specific method, a conflict emerges if Class D tries to inherit from both B and C simultaneously. Java cannot determine which structural path's logic to execute. Note: Java allows multiple inheritance exclusively via interfaces because they do not carry state duplication.

Q6. What is the difference between an Abstract Class and an Interface?

An **Abstract Class** can provide a mixture of implemented concrete methods and unimplemented abstract structures. It can hold custom instance variables, standard constructor blocks, and use any access modifier. A class can extend only one abstract block. An **Interface** traditionally served strictly as an absolute abstract template. Post Java 8, it can hold default and static concrete methods. They cannot feature stateful instance variables (only `public static final` constants) or constructor definitions.

Q7. What are default and static methods in interfaces?

Introduced in Java 8 to protect backwards compatibility:

- **Default methods** (`default`): Empower interfaces to introduce fully fleshed out method bodies without breaking preexisting legacy implementations across client systems. They can be safely overridden.
- **Static methods**: Bind directly to the enclosing interface type structure rather than concrete implementation entities. They cannot be overridden.

Q8. What is the difference between `this` and `super` keywords?

The reference `this` addresses the exact immediate running instance object context. It allows referencing internal state and constructor definitions. The reference `super` targets the directly adjacent parent class context, bypasses localized state masks, and links to super-constructors.

3. Constructors & Object Lifecycle

Q9. Can a constructor be overloaded? Can it be overridden?

Constructors can be extensively **overloaded** by varying the argument signatures inside the target class. However, they can never be **overridden** because constructors are not inherited by child classes.

Q10. What happens if you don't define a constructor in a class?

The compiler implicitly appends a hidden, parameterless **default constructor** to complete structural validation. However, if you explicitly implement *any* custom constructor variant (even with multi-arguments), the default implicit logic is permanently suppressed.

Q11. What is Constructor Chaining?

Constructor chaining is the programmatic sequence of calling consecutive constructor logic routines relative to one instantiation action. Local class definitions pass control using `this()`, while parent target paths pass initialization routines up the chain using `super()`.

4. Access Modifiers & Memory Keywords

Q12. Explain the different access modifiers in Java.

- **private:** Restricts operational context visibility strictly to the scope of the defining class block.
- **Default (no explicit keyword):** Limits complete runtime accessibility to packages matching the component layout.
- **protected:** Grants visibility inside the matching package space, alongside extended subclasses crossing external package lines via inheritance.
- **public:** Universally accessible across the global application layout space.

Q13. What is the use of the `final` keyword in Java?

The keyword enforces absolute immutability based on where it is applied:

- **Final Variable:** Transforms the target into a deterministic constant; values cannot be changed after assignment.
- **Final Method:** Locks execution implementation logic; child classes cannot override the method.
- **Final Class:** Cuts off structural extension entirely; the class cannot be inherited (e.g., the standard `String` class).

Q14. What is the static keyword?

Signals that a specific resource belongs natively to the **class template itself**, rather than allocating unique versions across concrete runtime object instances. Memory for static elements is allocated once in the specialized Metaspace area and shared uniformly.

Q15. Can you override a static method?

No. Overriding depends on dynamic runtime virtual table dispatching. Static methods rely entirely on early compile-time binding bound directly to class types. Redeclaring an identical signature simply performs **Method Hiding**, masking the superclass method without establishing polymorphic overriding behavior.

5. Advanced & Architectural Concepts

Q16. What is the difference between Composition and Inheritance?

Inheritance establishes a tight **Is-A** directional structural coupling (e.g., a **Car** is a **Vehicle**). **Composition** institutes a highly resilient, loosely coupled **Has-A** architectural composition structure (e.g., a **Car** has an **Engine**) by retaining an internal object reference instance variable.

Q17. What are Loose Coupling and High Cohesion?

Loose Coupling: Architectural design pattern optimizing individual parts so components change independently without rippling side-effects across the system (best achieved through Interface abstractions).

High Cohesion: Designing a targeted component or class module to perform exactly one specific, clearly focused responsibility exceptionally well without blending unrelated processing jobs.

Q18. What is object cloning? How do you achieve it?

The routine of building an exact duplicate runtime replica instance of an active object block. This requires implementing the **Cloneable** interface and declaring overrides for **Object.clone()**.

- **Shallow Copy:** Duplicates top-level reference values; nested sub-objects remain shared references across both instances.
- **Deep Copy:** Recursively creates completely separate allocations for nested objects, achieving absolute transactional independence.

Q19. Why is Java not considered a "Pure" Object-Oriented language?

A pure OOP language dictates that every standalone type must resolve explicitly as a complete object instance derived from a master class structure. Java fails pure status because it supports basic primitive types (like **int**, **char**, **float**, **boolean**) directly for architectural speed optimization.

Q20. What are the SOLID principles?

SOLID covers five foundational design directives engineered to make object-oriented architectures cleaner, flexible, and maintainable:

- **S - Single Responsibility:** A class must restrict its existence to exactly one clear reason to change.
- **O - Open/Closed:** Software systems must stand open for clean behavioral extension but strictly closed against modification.
- **L - Liskov Substitution:** Subtypes must be perfectly exchangeable for their parent base types without breaking code execution.
- **I - Interface Segregation:** Clients should never be forced to implement bloated interface contracts filled with methods they do not need.
- **D - Dependency Inversion:** Code architectures must rely heavily on abstract conceptual interfaces, rather than concrete execution layers.